

DeclarAI

Agentic RAG para apoio à declaração de IRPF



DeclarAI

ASSISTENTE FISCAL INTELIGENTE

Instituição: Universidade do Estado do Amazonas
Disciplina: Oficina e Desenvolvimento de Sistemas I
Projeto: Micro SaaS com Agentic RAG
Data: 7 de junho de 2026
Integrantes: Juliana Ballin Lima
Fernando Luiz Da Silva Freire

Manaus, AM

Sumário

1	Resumo Executivo	1
2	Matriz de Atendimento aos Requisitos	1
3	Definição do Problema	2
4	Arquitetura da Solução	2
4.1	Visão Geral	2
4.2	Pipeline RAG	3
4.3	Comportamento Agentic	3
4.4	Ferramentas do Agente	3
5	Base de Conhecimento	4
6	Modelo de Linguagem e Recuperação	4
6.1	Justificativa para Execução Local	4
6.2	Seleção dos Modelos	5
7	Interface	6
8	Diagramas e Identidade Visual	6
9	Avaliação e Comparações	7
9.1	Métricas implementadas	7
9.2	Comparação entre modelos LLM	7
9.3	Comparação de Estratégias de Chunking	8
9.4	Scripts de avaliação	9
10	Etapas de Desenvolvimento	9
11	Validação Executada	10
12	Divisão de Tarefas	11
13	Limitações e Riscos	11
14	Conclusão	11

1 Resumo Executivo

O DeclarAI é um assistente inteligente para apoiar contribuintes brasileiros na organização de documentos e no entendimento de regras da declaração do Imposto de Renda Pessoa Física. A solução utiliza uma arquitetura Agentic RAG com base de conhecimento própria, LLM aberto via Ollama, embeddings locais, ChromaDB, API FastAPI e frontend React.

O sistema permite consultar regras fiscais, enviar documentos, extrair dados, classificar categorias tributárias, verificar titularidade e gerar justificativas fundamentadas. A proposta não substitui a orientação profissional de um contador, mas reduz erros comuns e melhora a preparação documental do contribuinte.

2 Matriz de Atendimento aos Requisitos

Requisito da atividade	Status	Evidência no projeto
Problema e domínio	Atendido	O domínio é IRPF para contribuintes pessoas físicas, com foco em organização documental, deduções, titularidade e consulta a regras fiscais.
Base de conhecimento própria	Atendido	A base contém <code>guia_imposto_renda.txt</code> e <code>pr-irpf-2024.pdf</code> , ambos justificados por relevância fiscal.
Ingestão e pré-processamento	Atendido	O backend carrega PDF, TXT e HTML, limpa textos, preserva metadados e re-indexa a base pela API.
Chunking ou estruturação	Atendido	O padrão 600/80 foi escolhido após comparação de sete estratégias e está disponível para re-indexação experimental.
Embeddings e recuperação	Atendido	Usa embeddings <code>MiniLM</code> multilíngues, ChromaDB, busca vetorial e re-ranking com CrossEncoder.
Agentic RAG	Atendido	O agente escolhe ferramentas para chat, upload, classificação, titularidade, justificativa, histórico e avaliação.
LLM aberto	Atendido	O Mistral é executado localmente via Ollama, com justificativa de privacidade, custo e suporte a português.
Geração de respostas	Atendido	As respostas usam contexto recuperado, fontes, score médio e regra de indicar insuficiência quando necessário.

Requisito da atividade	Status	Evidência no projeto
Interface ou API	Atendido	O projeto possui frontend React, API FastAPI, Swagger, Docker Compose e comandos locais.
Avaliação da solução	Atendido	Há dataset com 60 perguntas, scripts de avaliação, comparação de modelos, avaliação de recuperação e avaliação completa.
Documentação e apresentação	Atendido	README, relatório, diagramas, roteiro de demonstração e prompt de slides documentam arquitetura, decisões, limites e uso.

3 Definição do Problema

A declaração do IRPF envolve regras, limites, documentos comprobatórios e categorias que mudam com o ano fiscal. Usuários leigos frequentemente enfrentam três dificuldades:

- identificar se uma despesa é dedutível;
- organizar documentos aceitos pela Receita Federal;
- entender quando uma informação está fora da base de conhecimento.

O público-alvo são contribuintes pessoas físicas com perfil simples ou intermediário, especialmente usuários que acumulam recibos, notas fiscais, informes de rendimentos e comprovantes educacionais ou médicos durante o ano.

4 Arquitetura da Solução

4.1 Visão Geral

Camada	Responsabilidade
Frontend React	Interface de chat, upload, base de conhecimento, histórico, avaliação de recuperação, avaliação completa e status.
FastAPI	Endpoints REST, validações, orquestração de serviços e documentação Swagger.
Pipeline RAG	Ingestão, chunking, embeddings, recuperação, re-ranking e geração.
Ollama	Execução local do LLM aberto usado em geração e classificação.
ChromaDB	Persistência vetorial dos chunks recuperáveis.
SQLite	Histórico de documentos salvos e dados estruturados.

4.2 Pipeline RAG

O pipeline segue oito etapas:

1. carregamento de documentos da base;
2. extração e limpeza textual;
3. divisão em chunks de 600 caracteres com 80 caracteres de sobreposição;
4. geração de embeddings multilíngues;
5. indexação no ChromaDB;
6. recuperação semântica dos trechos;
7. re-ranking com CrossEncoder;
8. geração de resposta com LLM local.

4.3 Comportamento Agentic

O agente decide qual ferramenta usar conforme a intenção do usuário. Perguntas abertas acionam busca vetorial. Uploads acionam extração, classificação, verificação de titularidade e justificativa enriquecida. Consultas sobre documentos salvos usam histórico e metadados.

4.4 Ferramentas do Agente

O sistema implementa seis ferramentas distintas, cada uma acionada de acordo com a natureza da entrada do usuário:

Ferramenta	Acionada quando	Funcionamento
Busca Vetorial	pergunta de texto livre	embedding da consulta, ChromaDB top-15, CrossEncoder top-5, geração com Mistral
Classificação LLM-first	upload de documento	LLM analisa categoria, fallback por palavras-chave; campo <code>origem_classificacao</code> indica o método usado
Extração de Dados	upload de documento	extraí emitente, CNPJ, valor, data e nome do beneficiário
Verificação de Titularidade	após classificação	compara beneficiário com perfil do declarante; retorna: titular, dependente provável ou terceiro
Justificativa Enriquecida	após classificação	segundo pipeline RAG: top-3 chunks por categoria, prompt com dados do documento, Mistral gera explicação normativa
Consulta ao Histórico	histórico e resumo anual	SQLite retorna documentos, totais estimados, alertas de limite e economia projetada

5 Base de Conhecimento

Documento	Tipo	Justificativa
guia_imposto_renda.txt	TXT	Reúne regras resumidas de obrigatoriedade, deduções e documentos comuns do IRPF.
pr-irpf-2024.pdf	PDF	Documento oficial de perguntas e respostas da Receita Federal, usado como fonte primária.

Essas fontes cobrem despesas médicas, educação, previdência privada, rendimentos, dependentes, aluguéis, prazos e penalidades. A base pode ser ampliada pela interface.

6 Modelo de Linguagem e Recuperação

6.1 Justificativa para Execução Local

Documentos fiscais contêm dados altamente sensíveis: CPF, renda anual, despesas médicas, informações de dependentes e valores de patrimônio. O envio desses dados para APIs externas de LLM em nuvem representaria risco à privacidade do contribuinte, possível violação da Lei Geral de Proteção de Dados (LGPD) e dependência de serviços pagos e de conectividade.

O uso do Ollama garante execução inteiramente local: nenhum dado sai do ambiente do usuário. Essa decisão elimina custos recorrentes por token, remove a dependência de internet e mantém pleno controle sobre o processamento das informações. Modelos em nuvem como GPT-4 e Gemini foram descartados por esses motivos, mesmo que pudessem oferecer maior capacidade de geração.

6.2 Seleção dos Modelos

Modelo de Geração e Classificação (LLM)

Quatro modelos foram avaliados via Ollama no dataset de 60 perguntas do domínio IRPF:

Modelo	Cobertura (%)	Latência (s)	Observação
Mistral + RAG (escolhido)	72,4	8,2	Maior cobertura; bom suporte ao PT-BR
Phi-4 Mini + RAG	68,3	7,0	Boa relação tamanho/qualidade
Llama 3.2:3b + RAG	65,8	6,4	Mais leve, menor cobertura
Gemma 3:4b + RAG	61,2	6,9	Menor cobertura no domínio fiscal

O Mistral foi escolhido por apresentar a maior cobertura de palavras-chave esperadas (72,4%) e desempenho consistente em perguntas sobre regras fiscais em português. Sua licença aberta e compatibilidade com Ollama também foram fatores determinantes.

Modelo de Embeddings

O `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` foi escolhido por três razões: (1) suporte nativo ao português brasileiro, sem necessidade de tradução prévia; (2) dimensionalidade de 384, adequada para ChromaDB em CPU; e (3) tamanho reduzido, permitindo execução eficiente em máquinas sem GPU.

Modelo de Re-ranking

O `cross-encoder/mmarco-mMiniLMv2-L12-H384-v1` foi selecionado por ser multilíngue e treinado em tarefas de recuperação de informação. Apresenta desempenho superior em perguntas com negações e exceções fiscais, como “não é dedutível” ou “apenas para autônomos”, casos em que a busca vetorial pura tende a retornar trechos irrelevantes.

Decisão	Motivo
Ollama local	Evita envio de dados fiscais sensíveis para APIs externas e garante conformidade com a LGPD.
Mistral	Maior cobertura no domínio fiscal entre os modelos testados; licença aberta.
MiniLM multilíngue	Leve, com suporte a PT-BR e compatível com CPU sem GPU.
CrossEncoder multilíngue	Re-ranking preciso em perguntas fiscais com negações e exceções.
ChromaDB	Banco vetorial simples, persistente e suficiente para protótipo local.
Temperatura baixa	Reduz alucinações em domínio fiscal com regras específicas.

7 Interface

O frontend foi desenvolvido em React + Vite. A interface possui páginas de uso direto:

- Chat RAG com fontes consultadas;
- Upload de documentos com perfil do declarante, verificação de titularidade e revisão antes de salvar;
- Base de conhecimento para adicionar, remover e re-indexar fontes;
- Histórico de documentos salvos, com resumo anual e economia estimada;
- Avaliação do pipeline, com modo de recuperação e modo completo com LLM;
- Status do sistema, incluindo Ollama, modelos e chunks indexados.

8 Diagramas e Identidade Visual

A documentação visual foi atualizada com fundo claro, textos acentuados e espaçamento suficiente para evitar sobreposição de setas e componentes. A nova marca do DeclarAI foi gerada em SVG e exportada para PNG, sendo usada no relatório e no frontend.

Arquivo	Conteúdo
docs/diagrams/logo.svg	Logo horizontal do DeclarAI.
docs/diagrams/logo_icon.svg	Ícone usado no frontend.
docs/diagrams/rag/recuperacao_reranking.svg	Fluxo de recuperação, re-ranking e geração da resposta.
docs/diagrams/agentik_rag/titularidade_justificativa.svg	Fluxo agentik de upload, titularidade, justificativa e salvamento.
docs/diagrams/rag/indexacao_e_ingestao.svg	Ingestão, limpeza, chunking, embeddings e ChromaDB.
docs/diagrams/agentik_rag/ferramentas_agente.svg	Ferramentas disponíveis ao agente.

9 Avaliação e Comparações

O dataset `data/eval/perguntas.json` possui 60 perguntas anotadas com resposta de referência, palavras-chave e nível de dificuldade distribuídos em 12 categorias fiscais: obrigatoriedade, deduções médicas, educação, previdência privada, rendimentos, prazos, penalidades, autônomos, dependentes, aluguéis, doações e documentos fiscais.

9.1 Métricas implementadas

- **Taxa de recuperação:** proporção de perguntas com ao menos um chunk recuperado;
- **Score médio de contexto:** média das similaridades cosseno dos chunks retornados;
- **Cobertura de palavras-chave:** percentual dos termos esperados encontrados na resposta gerada;
- **Latência por pergunta:** tempo médio de resposta em segundos;
- **Análise de falhas:** casos com cobertura abaixo de 50%.

9.2 Comparação entre modelos LLM

A tabela abaixo compara o modelo padrão do sistema (Mistral com RAG) contra variantes sem RAG e outros modelos disponíveis via Ollama, avaliados no dataset de 60 perguntas do domínio IRPF.

Configuração	Cobertura (%)	Latência (s)	Observação
Mistral + RAG (padrão)	72,4	8,2	Melhor cobertura; modelo padrão do sistema
Phi-4 Mini + RAG	68,3	7,0	Boa relação tamanho/qualidade
Llama 3.2:3b + RAG	65,8	6,4	Modelo mais leve
Gemma 3:4b + RAG	61,2	6,9	Menor cobertura no domínio fiscal
Mistral sem RAG	44,1	5,1	Ablation study; confirma necessidade do RAG

A diferença de cobertura entre o Mistral com RAG (72,4%) e sem RAG (44,1%) confirma que a recuperação semântica contribui com +28,3 pontos percentuais na qualidade das respostas sobre domínio fiscal, justificando a arquitetura RAG adotada.

9.3 Comparação de Estratégias de Chunking

Para definir a configuração de chunking mais adequada ao domínio fiscal, foram desenhadas e testadas sete configurações distintas, avaliadas com as métricas de recuperação do dataset anotado.

Configuração	Tamanho	Sobreposição	Observação
fixo_200_0	200	0	Trechos curtos; perde contexto de regras fiscais
fixo_400_40	400	40	Equilíbrio fraco; sobreposição insuficiente
fixo_600_80	600	80	Melhor equilíbrio: contexto fiscal + precisão semântica
fixo_800_120	800	120	Contexto rico, mas maior ruído semântico nos embeddings
fixo_1000_200	1000	200	Trechos longos demais; diluem a similaridade vetorial
sentença	variável	0	Fragmentos irregulares em documentos fiscais
semântico	variável	variável	Experimental; sem vantagem clara no domínio

A configuração 600/80 foi selecionada por dois motivos principais: (1) preserva parágrafos completos de regras fiscais, incluindo exceções e limites de dedução; (2) a sobreposição de 80 caracteres mantém a continuidade de sentido entre fragmentos adjacentes sem aumentar demasiadamente o volume de tokens enviados ao LLM.

9.4 Scripts de avaliação

Script	Objetivo
scripts/avaliar_llm.py	Comparar modelos Ollama e executar ablation study sem RAG.
scripts/avaliar_chunking.py	Comparar configurações de chunking e latência.
scripts/avaliar_ragas.py	Calcular métricas RAGAS com Ollama local como juiz.

Os resultados são salvos em `data/eval/resultados_llm.csv` para análise e reprodução.

10 Etapas de Desenvolvimento

Etapa	Status	Resultado
Base RAG	Concluída	Ingestão, chunking, ChromaDB, embeddings e resposta contextualizada.
Re-ranking semântico	Concluída	CrossEncoder multilíngue melhora a ordem dos trechos recuperados.
Classificação LLM-first	Concluída	Pipeline com fallback por regras e origem da classificação.
Justificativa enriquecida	Concluída	Segundo pipeline RAG gera explicação fundamentada na base.
Deteção de titularidade	Concluída	Identifica titular, dependente provável e divergências de nome.
Frontend React	Concluída	Interface profissional com chat, upload, histórico e avaliação.
Dataset de avaliação	Concluída	60 perguntas anotadas com keywords, resposta e dificuldade.
Comparação de modelos	Concluída	Scripts salvam resultados em CSV para análise comparativa.
Relatório LaTeX	Concluída	Documentação técnica com tabelas de avaliação.
Validação técnica	Concluída	Testes de smoke, build do frontend e validações de interface executados.
Busca híbrida	Futuro	BM25 + vetorial + fusão de rankings (RRF).
Notebooks analíticos	Futuro	Visualizações para artigo e apresentação.

11 Validação Executada

A validação final combinou testes automatizados, build de interface, checagens Docker e fluxos HTTP dos principais casos de uso.

Validação	Resultado	Observação
<code>pytest -v</code>	Aprovado	6 testes passaram e 1 teste de extração completa foi pulado na <code>venv</code> leve por ausência de <code>pdfplumber</code> .
<code>make check</code>	Aprovado	<code>isort</code> , <code>black</code> e <code>flake8</code> sem pendências no backend.
<code>npm run build</code>	Aprovado	Build Vite gerado sem erro.
<code>npm run lint</code>	Aprovado	ESLint sem avisos ou erros.
Docker Compose	Aprovado	Backend e frontend saudáveis, Ollama disponível e modelo <code>mistral</code> instalado.
Fluxo de upload	Aprovado	Recibo odontológico de dependente processado com beneficiário correto, categoria <code>Recibo Médico</code> e justificativa enriquecida.
Histórico	Aprovado	Documento salvo, listado no histórico, considerado no resumo anual e removido em seguida no teste.
Chat RAG	Aprovado	Pergunta sobre curso de inglês recuperou 5 trechos e respondeu corretamente que não é dedutível.
Avaliação de recuperação	Aprovado	Execução pela API retornou recuperação em todos os casos testados.

12 Divisão de Tarefas

Integrante	Responsabilidades principais
Juliana Ballin Lima	Arquitetura do pipeline RAG, re-ranking com CrossEncoder, classificação LLM-first, verificação de titularidade, frontend React (design system, componentes e páginas), diagramas e relatório técnico.
Fernando Luiz Da Silva Freire	Ingestão e pré-processamento de documentos, extração de dados fiscais, serviço de justificativa enriquecida, dataset de avaliação, scripts de comparação de modelos e configuração Docker/CI.
Compartilhado	Definição dos requisitos, base de conhecimento, testes de integração, revisão de código e apresentação.

13 Limitações e Riscos

- O sistema depende do Ollama e dos modelos instalados localmente.
- Regras fiscais mudam anualmente e exigem atualização da base.
- OCR pode falhar em imagens de baixa qualidade.
- A classificação pode exigir revisão do usuário em documentos atípicos.
- O projeto é apoio informativo e não substitui contador.

14 Conclusão

O DeclarAI atende a todos os requisitos da atividade: domínio bem definido (IRPF), base de conhecimento própria com documentos oficiais da Receita Federal, ingestão e pré-processamento com limpeza textual, chunking com sobreposição, embeddings multilíngues, recuperação semântica com re-ranking, LLM aberto via Ollama, comportamento Agentic RAG com seleção de ferramentas, interface web profissional e avaliação quantitativa com dataset anotado.

A comparação entre modelos mostra que o RAG contribui com ganho médio de 28 pontos percentuais na cobertura de keywords em relação ao LLM sem recuperação, validando a escolha arquitetural. A evolução futura prioritária é a busca híbrida (BM25 + vetorial) e a expansão da base com documentos atualizados para o exercício fiscal vigente.